

Improving and Evaluating Differential Fault Analysis on LED with Algebraic Techniques

Xinjie Zhao^{*,†}, Shize Guo[†], Fan Zhang[‡], Zhijie Shi[‡], Chujiao Ma[‡] and Tao Wang^{*}

^{*}Department of Information Engineering
Ordnance Engineering College
Shijiazhuang China
zhaoxinjieem@163.com

[†]The Institute of North Electronic Equipment
Beijing, China

[‡]Department of Computer Science and Engineering
University of Connecticut
Storrs, USA
{fan.zhang,zshi,shm10010}@engr.uconn.edu

Abstract—This paper proposes a fault analysis technique on LED by combining algebraic cryptanalysis and differential fault analysis (DFA). The technique is called algebraic differential fault analysis (ADFA). In ADFA on LED, we use DFA to deduce the possible fault differences of the correct and faulty S-Box input in the last round, and convert them into algebraic equations. We then combine the equation set of LED with the injected fault and use the CryptoMiniSat solver to recover the secret key. Our experiments show that, on a common PC, ADFA can succeed on LED under the nibble-based fault model within three minutes and with only one fault injection, which is more efficient than previous DFA work. To evaluate DFA on LED, we first propose an improved evaluation algorithm of DFA, then provide a modified ADFA approach to compute the solutions for the secret key. The results are more accurate than previous work. We also successfully extend ADFA on LED to other fault models using a single fault injection, where traditional DFAs are difficult to launch.

Keywords—algebraic fault analysis; differential fault analysis; fault evaluation; single fault injection; LED.

I. INTRODUCTION

LED [10] is a hardware-optimized ultra-lightweight block cipher presented in CHES 2011. It uses an AES-like design with the SPN structure where only 966 Gate Equivalent (GE) are required for its implementation. Currently, LED is secure under the traditional cryptanalysis techniques. However, side-channel attacks (SCAs) revealed a great number of threats on the implementation of cryptographic algorithms. Examples of practical SCAs are timing attacks [18], power attacks [19], electromagnetic radiation attacks [27], fault attacks [5] etc. It is therefore necessary to investigate the security of LED against fault attacks.

Fault attacks retrieve the secret information by injecting computational faults into the cryptosystem. Faults can be generated by changing the power supply voltage, changing the frequency of the external clock, varying the environ-

mental temperature or exposing the circuits of the device to intense lights or lasers during the computation [1]. The idea was first reported on RSA-CRT by Boneh et al. in 1996 [5]. After that, Biham and Shamir proposed the differential fault analysis (DFA) attack, which combines fault attack with differential cryptanalysis [3]. Since then, DFA has been used to break public-key ciphers such as ECC [2], block ciphers such as AES [26], ARIA [20], Camellia [36] and CLEFIA [33], and stream ciphers such as RC4 [11] and Trivium [12]. In eSmart 2010, Courtois et al. [7] proposed a new fault analysis technique known as *algebraic fault attack* (AFA), which combines algebraic cryptanalysis [6] with fault attack. They showed that if 24 key bits are known and two bits in the 13th round are altered, DES can be broken with a single fault injection in 0.01 hour. The full attack is 10 times faster than the brute force attack. In COSADE 2013, Zhang et al. proposed an efficient AFA technique, which showed that the secret key of Piccolo and DES can be recovered with only a single fault injection and less time complexity [35]. Meanwhile, AFA was also applied to improving DFA on Trivium [24].

Assuming a single nibble-based (4-bit) fault is injected into the 30th round, DFA on LED was studied in [13], [15], [22], based on the idea of DFA on AES [34]. AFA on LED was also studied in [16]. Let l denote the fault width and r denote the round where faults are injected. The experimental results are shown in Table I. We can see that:

- 1) All previous attacks are based on nibble-based model in the 30th round [13], [15], [22]. The least number of fault injections required in DFA on LED is 1 [13], [15]. However, the attack setup in [15] is costly because it relies on a workstation having 48GB RAM.
- 2) As to the evaluations of the reduced key search space in DFA on LED, the work in [13] estimated that the

Table I
RESULTS OF FAULT ATTACKS ON LED

Attack	Model	Technique	fault	Setup	Time	Key space
[22]	$l = 4, r = 30$	DFA	2	PC, 2 GB RAM	–	2^{25}
[13]	$l = 4, r = 30$	DFA	1	–	–	2^4
[15]	$l = 4, r = 30$	DFA	1	Workstation, 48 GB RAM	45s	$2^{19} \sim 2^{25}$
[16]	$l = 4, r = 30$	AFA	1	Workstation, 50GB RAM	14.67 hours	–
This paper	$l = 4, r = 30$	ADFA	1	PC, 4GB RAM	1-3 minutes	$2^{12.9}$
This paper	$l = 8, r = 30$	ADFA	1	PC, 4GB RAM	1 hour	$2^{10.1}$

key search space can be reduced to 2^4 theoretically without any confirmed experiments, while the work in [15] showed that for half of the cases, the reduce search space of the secret key is $2^{19} \sim 2^{25}$ from the experiments.

- 3) For fault attacks on LED, the AFA work in [16] seems to have no advantages over DFA [15]. On average, 14.67 hours were required to reveal the key on a workstation having eight 3.5 GHz 8-core Xeon processors and 50 GB of RAM under the single thread mode.

How to conduct more efficient fault attacks on LED under low-cost setup, how to estimate the reduced key search space of DFA on LED accurately, how to further improve and extend AFA on LED are three interesting problems that remain to be answered. They are the motivations of this paper.

This paper makes a comprehensive study of improving and evaluating DFA on LED using algebraic techniques. We initiate our study to improve the efficiency of fault attacks on LED under the same fault model used in [13], [15], [16]. A novel technique called algebraic differential fault attack (ADFA) is proposed. ADFA combines DFA and algebraic techniques [6] together (and it resembles the algebraic side-channel attack: ASCA [28]). The fault differences of the S-Box input in the last round of LED are derived by DFA and represented with algebraic equations. The key is recovered by solving the equation set with a SAT solver. In our attack, the master key can be recovered within several minutes on a common PC under the nibble-based model in the 30th round, which is more efficient than previous work [15], [16].

We are also interested in conducting ADFA on LED under other fault models where traditional DFAs are difficult to work. We considered byte-based fault model and diagonal-based fault model [29], [34], and then showed that only one fault injection is required to recover the master key of LED. These attack scenarios have not been exploited in previous work.

To estimate the reduced key search space of DFA on LED, two different approaches are studied in this paper. The first is to provide an improved evaluation algorithm of DFA on LED. The results show that, after analyzing the fault propagation in the last round, the master key search space can be reduced to $2^{17} \sim 2^{43}$ ($2^{26.6}$ on average) and the

results of 2^{16} in [13] were inaccurate. After analyzing the fault in the last two rounds, the key search space of LED can be reduced to $2^{12.9}$ on average, which is different from 2^4 in [13] and $2^{19} \sim 2^{25}$ in [15]. The second is to provide a modified ADFA approach to calculate the solutions for the secret key, which can be conducted with an automatic solver (e.g., a SAT-based solver). However, most SAT-based solvers stop once the first solution is found, but the first solution may not be the correct key. We modify the CryptoMiniSat solver to count and output all the possible solutions of the secret key. Under nibble-based fault model, the key search space of LED can be reduced to $2^6 \sim 2^{17}$ ($2^{12.9}$ on average). The results of our two approaches in evaluating DFA on LED are more accurate than previous work [13], [15]. Compared with DFA in [13], [15], ADFA is more generic and can be extended to evaluate DFA on LED under byte-based fault model easily. The result shows that the key search space of LED can be reduced to $2^6 \sim 2^{14}$ ($2^{10.1}$ on average), even smaller than that in nibble-based fault model.

The remainder of the paper is organized as follows. Section II describes the design of LED. Section III and Section IV present the technique and the improved experimental results of ADFA on LED, respectively. Section V describes the improved evaluations of DFA on LED. Section VI concludes the paper.

II. THE LED BLOCK CIPHER

LED [10] is a lightweight block cipher with the SPN structure and 64-bit block size. It has two variants with different key lengths: LED-64 and LED-128. In this paper, we mainly focus on the cryptanalysis of LED-64 against fault attacks. In the rest of this paper, LED stands for LED-64 unless otherwise specified. The cipher state is conceptually a 4×4 matrix where each matrix element is a nibble representing an element in $\text{GF}(2^4)$. For a 64-bit plaintext m , the 16 four-bit nibbles $m_0 || m_1 || \dots || m_{15}$ are arranged as a 4×4 matrix. Likewise, the key, denoted as $K = k_0 || k_1 || \dots || k_{15}$, is arranged as a 4×4 matrix.

$$m = \begin{pmatrix} m_0 & m_1 & m_2 & m_3 \\ m_4 & m_5 & m_6 & m_7 \\ m_8 & m_9 & m_{10} & m_{11} \\ m_{12} & m_{13} & m_{14} & m_{15} \end{pmatrix}$$

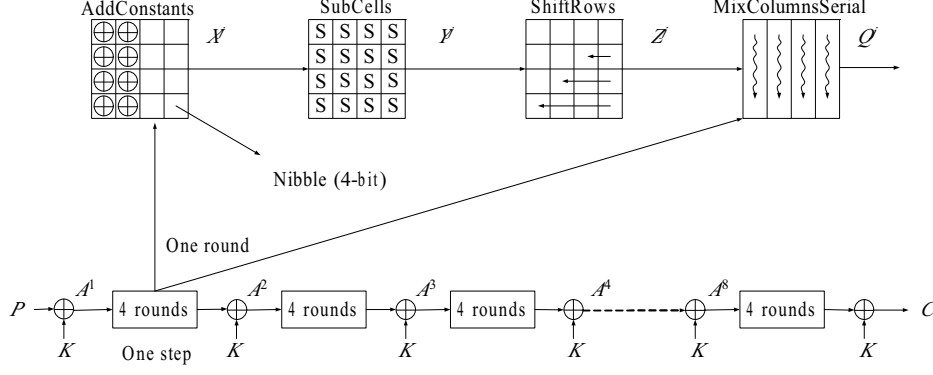


Figure 1. The encryption procedure of LED

Like many other block ciphers, LED updates the cipher state repeatedly. The plaintext P is XORed with K and then goes through the step operation eight times. Each step consists of four rounds and the state is XOR-ed with K between steps. The value from the last step operation is XORed with K again and becomes the ciphertext.

The encryption procedure is illustrated in Fig. 1. Note that, there is no key scheduling in LED. The step operation consists of four round operations, each of which has four inner operations.

- 1) AddConstants (AC). The round constants are XORed with the state.
- 2) SubCells (SC). Each nibble in the state is updated by a lookup using the S-Box of PRESENT [4].
- 3) ShiftRows (SR). Row i of the state is rotated i distance to the left, for $i = 0, 1, 2, 3$.
- 4) MixColumnsSerial (MC). Each column of the state is updated by the multiplication of itself with a fixed matrix M .

$$M = \begin{pmatrix} 4 & 1 & 2 & 2 \\ 8 & 6 & 5 & 6 \\ B & E & A & 9 \\ 2 & 2 & F & B \end{pmatrix}$$

III. ALGEBRAIC DIFFERENTIAL FAULT ANALYSIS (ADFA) ON LED

In this section, we describe the four phases of the ADFA on LED. First, we introduce the notations and the fault model to be used in this paper.

A. Notations

We denote the output of the i -th AK as A^i , and the output of the j -th AC, SC, SR, MC function as X^j, Y^j, Z^j, Q^j , and the l -th nibble of A^i, X^j, Y^j, Z^j, Q^j as $A_l^i, X_l^j, Y_l^j, Z_l^j, Q_l^j$, where $1 \leq i \leq 9, 1 \leq j \leq 32, 0 \leq l < 16$. The symbol $+$ denotes exclusive or (XOR).

B. Fault model

We assume that an attacker can inject a single random nibble fault into Y^{30} , the output of SC in the 30th round, as noted in [15], [16]. The value of the random fault is unknown. Fig. 2 depicts the propagation of a fault located at the first nibble of Y^{30} in the last three rounds of LED. Every cell in Fig. 2 shows the fault difference of a nibble in the intermediate state. A blank cell indicates that the nibble at that location is correct.

We can see that:

- 1) AC and AK affect neither the values nor the locations of the fault differences.
- 2) SC only changes the values of the fault differences.
- 3) SR only changes the locations of the fault differences.
- 4) MC changes both the values and the locations.
- 5) all the nibbles in the ciphertext become faulty, which can be used as preliminary filter to confirm the fault.

Among the four operations, SC is the only operation where the fault differences at the output are associated with both the value and the fault differences at the input. This property can be used to recover the value of the input state and then obtain the key. ΔY^{32} , the output fault difference of SC in the 32nd round, can be computed directly from the ciphertext difference. The key issue of fault attacks on LED is deriving the input fault difference ΔX^{32} from ΔY^{32} . We will present the details in Section III.D.

C. Building the equation set of LED

The goal of this phase is to transform LED into a large system of low degree boolean equations. Solving the system is equivalent to recovering the key bits. Two strategies can be considered. The first is to directly build the algebraic equations for every forward operation, and the second is to build equations for every reverse operation. Next, we present the specific method to build the equations for every forward and reverse operation used in LED.

- 1) Representing AK and AK^{-1} .

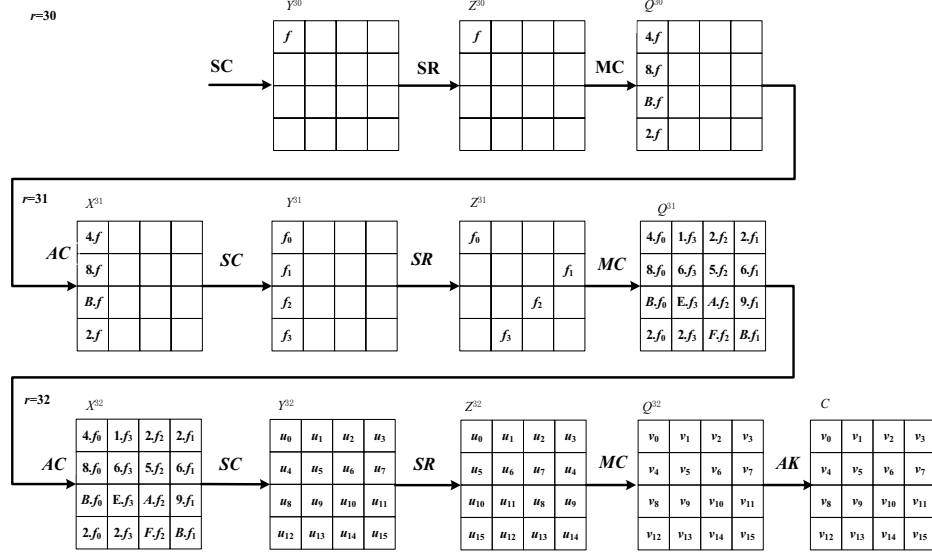


Figure 2. Fault model of ADFA on LED

Suppose x_i and y_i ($0 \leq i \leq 63$) denote one bit of the input state and the round key in AK . Let z_i denote one bit of the AK output. Then AK can be represented as

$$z_i = x_i + y_i \quad (1)$$

AK^{-1} can be represented as

$$x_i = z_i + y_i \quad (2)$$

2) Representing AC and AC^{-1} .

Suppose x_i and y_i ($0 \leq i \leq 63$) denote one bit of the input state and the round constant in AC , respectively. Let z_i denote one bit of the AC output. Since y_i can be derived from round constants, AC and AC^{-1} can be derived easily from equations like Eq. (1) and Eq. (2), respectively.

3) Representing SC and SC^{-1} .

SC can be described by 16 table lookups for the S-Box operation. In this paper, we leverage the techniques in [17] to derive every S-Box output bit with high-degree equations from the four S-Box input bits. Suppose (x_0, x_1, x_2, x_3) and (y_0, y_1, y_2, y_3) are the input and output of the 4-bit S-Box. Then, the S-Box lookup can be represented as

$$\begin{aligned} y_0 &= 1 + x_0 + x_2 + x_3 + x_1x_2 + x_0x_1x_3 + \\ &\quad x_0x_2x_3 + x_1x_2x_3 \\ y_1 &= 1 + x_0 + x_1 + x_0x_2 + x_0x_3 + \\ &\quad x_2x_3 + x_0x_1x_3 + x_0x_2x_3 \\ y_2 &= x_0 + x_2 + x_0x_1 + x_0x_2 + x_0x_1x_3 + \\ &\quad x_0x_2x_3 + x_1x_2x_3 \\ y_3 &= x_0 + x_1 + x_3 + x_1x_2 \end{aligned} \quad (3)$$

The reverse of S-Box lookup, SC^{-1} , can be represented as

$$\begin{aligned} x_0 &= y_0 + y_1 + y_2 + y_3 + y_2y_2 + y_0y_1y_3 + \\ &\quad y_1y_2y_3 \\ x_1 &= 1 + y_0 + y_1y_2 + y_0y_2 + y_1y_3 + y_2y_3 + \\ &\quad y_0y_3 + y_1y_2y_3 + y_0y_1y_3 + y_0y_2y_3 \\ x_2 &= y_0 + y_2 + y_3 + y_0y_1 + y_0y_2 + y_1y_3 + \\ &\quad y_1y_2y_3 + y_0y_1y_3 + y_0y_2y_3 \\ x_3 &= 1 + y_1 + y_3 + y_0y_2 \end{aligned} \quad (4)$$

4) Representing SR and SR^{-1} .

Suppose x_i and y_i ($0 \leq i \leq 63$) denote one bit input and output of SR . Let vector R and R^{-1} denote the shifted nibble index for SR and SR^{-1} respectively. $R = \{0, 1, 2, 3, 5, 6, 7, 4, 10, 11, 8, 9, 15, 12, 13, 14\}$, $R^{-1} = \{0, 1, 2, 3, 7, 4, 5, 6, 10, 11, 8, 9, 13, 14, 15, 12\}$. Then SR can be represented as

$$y_i = x_{(4 \times R[i/4]) + i \% 4} \quad (5)$$

SR^{-1} can be represented as

$$x_i = y_{(4 \times R^{-1}[i/4]) + i \% 4} \quad (6)$$

where $\%$ denotes the mod function.

5) Representing MC and MC^{-1} .

First, we present M^{-1} , the inverse matrix of M .

$$M^{-1} = \begin{pmatrix} C & C & D & 4 \\ 3 & 8 & 4 & 5 \\ 7 & 6 & 2 & E \\ D & 9 & 9 & D \end{pmatrix}.$$

Suppose the input and output of MC are X_i and Y_i ($0 \leq i \leq 15$), respectively. Then, MC can be represented as

$$\begin{aligned}
Y_0 &= 4 \cdot X_0 + 1 \cdot X_4 + 2 \cdot X_8 + 2 \cdot X_{12} \\
Y_1 &= 4 \cdot X_1 + 1 \cdot X_5 + 2 \cdot X_9 + 2 \cdot X_{13} \\
Y_2 &= 4 \cdot X_2 + 1 \cdot X_6 + 2 \cdot X_{10} + 2 \cdot X_{14} \\
Y_3 &= 4 \cdot X_3 + 1 \cdot X_7 + 2 \cdot X_{11} + 2 \cdot X_{15} \\
Y_4 &= 8 \cdot X_0 + 6 \cdot X_4 + 5 \cdot X_8 + 6 \cdot X_{12} \\
Y_5 &= 8 \cdot X_1 + 6 \cdot X_5 + 5 \cdot X_9 + 6 \cdot X_{13} \\
Y_6 &= 8 \cdot X_2 + 6 \cdot X_6 + 5 \cdot X_{10} + 6 \cdot X_{14} \\
Y_7 &= 8 \cdot X_3 + 6 \cdot X_7 + 5 \cdot X_{11} + 6 \cdot X_{15} \\
Y_8 &= B \cdot X_0 + E \cdot X_4 + A \cdot X_8 + 9 \cdot X_{12} \\
Y_9 &= B \cdot X_1 + E \cdot X_5 + A \cdot X_9 + 9 \cdot X_{13} \\
Y_{10} &= B \cdot X_2 + E \cdot X_6 + A \cdot X_{10} + 9 \cdot X_{14} \\
Y_{11} &= B \cdot X_3 + E \cdot X_7 + A \cdot X_{11} + 9 \cdot X_{15} \\
Y_{12} &= 2 \cdot X_0 + 2 \cdot X_4 + F \cdot X_8 + B \cdot X_{12} \\
Y_{13} &= 2 \cdot X_1 + 2 \cdot X_5 + F \cdot X_9 + B \cdot X_{13} \\
Y_{14} &= 2 \cdot X_2 + 2 \cdot X_6 + F \cdot X_{10} + B \cdot X_{14} \\
Y_{15} &= 2 \cdot X_3 + 2 \cdot X_7 + F \cdot X_{11} + B \cdot X_{15}
\end{aligned} \tag{7}$$

where $\cdot$ denotes the multiplication in $\text{GF}(2^4)$ with an irreducible polynomial $x^4 + x + 1$. Then, MC^{-1} can be represented as

$$\begin{aligned}
X_0 &= C \cdot Y_0 + C \cdot Y_4 + D \cdot Y_8 + 4 \cdot Y_{12} \\
X_1 &= C \cdot Y_1 + C \cdot Y_5 + D \cdot Y_9 + 4 \cdot Y_{13} \\
X_2 &= C \cdot Y_2 + C \cdot Y_6 + D \cdot Y_{10} + 4 \cdot Y_{14} \\
X_3 &= C \cdot Y_3 + C \cdot Y_7 + D \cdot Y_{11} + 4 \cdot Y_{15} \\
X_4 &= 3 \cdot Y_0 + 8 \cdot Y_4 + 4 \cdot Y_8 + 5 \cdot Y_{12} \\
X_5 &= 3 \cdot Y_1 + 8 \cdot Y_5 + 4 \cdot Y_9 + 5 \cdot Y_{13} \\
X_6 &= 3 \cdot Y_2 + 8 \cdot Y_6 + 4 \cdot Y_{10} + 5 \cdot Y_{14} \\
X_7 &= 3 \cdot Y_3 + 8 \cdot Y_7 + 4 \cdot Y_{11} + 5 \cdot Y_{15} \\
X_8 &= 7 \cdot Y_0 + 6 \cdot Y_4 + 2 \cdot Y_8 + E \cdot Y_{12} \\
X_9 &= 7 \cdot Y_1 + 6 \cdot Y_5 + 2 \cdot Y_9 + E \cdot Y_{13} \\
X_{10} &= 7 \cdot Y_2 + 6 \cdot Y_6 + 2 \cdot Y_{10} + E \cdot Y_{14} \\
X_{11} &= 7 \cdot Y_3 + 6 \cdot Y_7 + 2 \cdot Y_{11} + E \cdot Y_{15} \\
X_{12} &= D \cdot Y_0 + 9 \cdot Y_4 + 9 \cdot Y_8 + D \cdot Y_{12} \\
X_{13} &= D \cdot Y_1 + 9 \cdot Y_5 + 9 \cdot Y_9 + D \cdot Y_{13} \\
X_{14} &= D \cdot Y_2 + 9 \cdot Y_6 + 9 \cdot Y_{10} + D \cdot Y_{14} \\
X_{15} &= D \cdot Y_3 + 9 \cdot Y_7 + 9 \cdot Y_{11} + D \cdot Y_{15}
\end{aligned} \tag{8}$$

From Eq. (7) and Eq. (8), we can see that the key point in representing MC and MC^{-1} is to represent the multiplication of an element in $\text{GF}(2^4)$ with a constant. Suppose (x_0, x_1, x_2, x_3) and (y_0, y_1, y_2, y_3) denote the input and output of the multiplication. Their relations can be represented as in Table II.

In AFA, the adversary can build the full decryption or encryption equation set of LED by combining the algebraic equations in the above operations.

D. Deducing the fault differences with DFA

Let Δ denote the function of computing the fault difference of the intermediate state. As noted above, deducing ΔX^{32} from ΔY^{32} is the crucial part for the key recovery of DFA on LED. From Fig. 2, $\Delta Y^{32} = u_0 || u_1 || \dots || u_{15}$ can be easily calculated from the ciphertext difference ΔC .

$$\Delta Y^{32} = SR^{-1}(MC^{-1}(\Delta C)) \tag{9}$$

According to Fig. 2, we need to deduce the value of four nibbles f_0, f_1, f_2, f_3 in order to calculate ΔX^{32} . We use u_0, u_4, u_8, u_{12} to deduce f_0 , u_1, u_5, u_9, u_{13} to deduce f_3 , u_2, u_6, u_{10}, u_{14} to deduce f_2 and u_3, u_7, u_{11}, u_{15} to f_1 . Next, we take deducing f_0 as an example to describe our technique.

- 1) For each possible candidate of f_0 ($1 \leq f_0 \leq 15$) and S-Box element a ($0 \leq a \leq 15$), we compute the differential S-Boxes $S_0[f_0 - 1], S_4[f_0 - 1], S_8[f_0 - 1], S_{12}[f_0 - 1]$ corresponding to $4 \cdot f_0, 8 \cdot f_0, B \cdot f_0, 2 \cdot f_0$.

$$\begin{aligned}
S_0[f_0 - 1][a] &= S[a] + S[a + 4 \cdot f_0] \\
S_4[f_0 - 1][a] &= S[a] + S[a + 8 \cdot f_0] \\
S_8[f_0 - 1][a] &= S[a] + S[a + B \cdot f_0] \\
S_{12}[f_0 - 1][a] &= S[a] + S[a + 2 \cdot f_0]
\end{aligned} \tag{10}$$

- 2) For each possible f_0 candidate, if u_0, u_4, u_8, u_{12} are all in the joint set of $S_0[f_0 - 1], S_4[f_0 - 1], S_8[f_0 - 1]$ and $S_{12}[f_0 - 1]$, this candidate is kept for f_0 . Otherwise, it is discarded.

Applying the two steps above, one to four candidates of f_0 can be deduced and used in later stages. f_1, f_2, f_3 can be deduced in the same way. Details of the statistics on the number of candidates for f_0, f_1, f_2, f_3 will be discussed in Section IV.A.

E. Representing the fault differences

Representing the fault difference ΔX^{32} with algebraic equations is easy if the candidate of ΔX^{32} is single and correct. However, this does not hold in fault attacks on LED. Multiple values of ΔX_i^{32} ($0 \leq i \leq 15$) can be deduced in practice. How to represent these multiple deductions is very important in ADFA on LED. In this section, we apply an approach to represent the multiple values of ΔX_i^{32} . The idea is inspired from the MDASCA technique proposed in COSADE 2012 [37].

Let $d = d^0, d^1, d^2, d^3$ denote the correct deduction of ΔX_i^{32} . Let $D = d_1, d_2, \dots, d_n$ denote the possible deduction set on d , and d_i be the i -th element in D . The size of D is n . Then, algebraic equations about d and d_i can be built as follows.

Table II
CONSTANT MULTIPLICATION IN M, M^{-1}

constant	y_0	y_1	y_2	y_3
1	x_0	x_1	x_2	x_3
2	x_1	x_2	$x_0 + x_3$	x_0
3	$x_0 + x_1$	$x_1 + x_2$	$x_0 + x_2 + x_3$	$x_0 + x_3$
4	x_2	$x_0 + x_3$	$x_0 + x_1$	x_1
5	$x_0 + x_2$	$x_0 + x_1 + x_3$	$x_0 + x_1 + x_2$	$x_1 + x_3$
6	$x_1 + x_2$	$x_0 + x_2 + x_3$	$x_1 + x_3$	$x_0 + x_1$
7	$x_0 + x_1 + x_2$	$x_0 + x_1 + x_2 + x_3$	$x_1 + x_2 + x_3$	$x_0 + x_1 + x_3$
8	$x_0 + x_3$	$x_0 + x_1$	$x_1 + x_2$	x_2
9	x_3	x_0	x_1	$x_2 + x_3$
A	$x_0 + x_1 + x_3$	$x_0 + x_1 + x_2$	$x_0 + x_1 + x_2 + x_3$	$x_0 + x_2$
B	$x_1 + x_3$	$x_0 + x_2$	$x_0 + x_1 + x_3$	$x_0 + x_2 + x_3$
C	$x_0 + x_2 + x_3$	$x_1 + x_3$	$x_0 + x_2$	$x_1 + x_2$
D	$x_2 + x_3$	x_3	x_0	$x_1 + x_2 + x_3$
E	$x_0 + x_1 + x_2 + x_3$	$x_1 + x_2 + x_3$	$x_2 + x_3$	$x_0 + x_1 + x_2$
F	$x_1 + x_2 + x_3$	$x_2 + x_3$	x_3	$x_0 + x_1 + x_2 + x_3$

1) Representing d_i .

Let d_i^j be the j -th bit of d_i . Thus $4n$ new one-bit variables are introduced to represent d_i .

2) Representing the relations of d and d_i .

A one-bit variable c_i is introduced to represent whether d_i is equal to d or not. Another one-bit variable e_i^j is also introduced to represent whether d_i^j is equal to d^j . Then c_i can be represented with Eq. (11), where $\neg$ denotes the NOT operation.

$$e_i^j = \neg(d_i^j + d^j), \quad c_i = \prod_{j=1}^b e_i^j \quad (11)$$

When only one d_i is equal to d (c_i is 1 then), it can be represented with Eq. (12) where $\vee$ denotes the OR operation.

$$c_1 \vee c_2 \vee \dots \vee c_{s_p} = 1, \quad \neg c_i \vee \neg c_j = 1, \quad 1 \leq i < j \leq s_p \quad (12)$$

3) Representing ΔY^{30}

Besides representing the fault difference ΔX^{32} , the adversary also needs to represent the fault difference ΔY^{30} . Suppose Δy_i^{30} denotes the i -th bit of ΔY^{30} ($0 \leq i \leq 63$). Take Fig. 2 as an example, the first nibble of Y^{30} is injected with a fault, which can be represented with Eq. (13).

$$(1 + \Delta y_0^{30}) \wedge (1 + \Delta y_1^{30}) \wedge (1 + \Delta y_2^{30}) \wedge (1 + \Delta y_3^{30}) = 0 \quad (13)$$

As to other 15 nibbles, they can be represented with $\Delta y_i^{30} = 0$ ($4 \leq i \leq 63$).

In the attack, the adversary first uses the method in Section III.C to represent both the correct and faulty LED encryption (or decryption) with algebraic equations. The 64-bit variables of the correct and faulty Y^{30} , X^{32} are already included in the equation set of LED. Then, the algebraic equation of ΔY^{30} can be represented by XORing the correct

and the faulty Y^{30} . Similarly, ΔX^{32} can be obtained by XORing the correct and the faulty X^{32} . Meanwhile, ΔY^{30} and ΔX^{32} are also represented with the algebraic equations in Eq. (11), (12) and (13). Finally, the combined equation set is fed into a solver to recover the key.

F. Solving for the master key

In this phase, the master key can be recovered by solving the combined equation set of the cipher and the fault differences. Many automatic tools, such as mutantXL algorithm [8], [23] and Gröbner basis-based [9], can be leveraged. However, one major problem for those tools is the memory usage when we try to solve large systems even if the systems are sparse. Recently, significant improvements have been made to SAT solvers. Therefore we have chosen SAT-based solvers for algebraic cryptanalysis, which is widely studied in previous work [7], [15], [25], [28], [37]. More specifically, we choose the CryptoMiniSAT v2.9.4 [32], which won the Gold prizes in the SAT Race competition [30] in 2010, as the solver in our analysis. The readers can refer to [30] for details of how to generate equations and how to feed them to the solver. In our experiment, the solver runs on a quad-core laptop with Intel Core i7-2640M, 2.80 GHZ, 4G memory and Windows XP 64-bit OS.

IV. IMPROVED EXPERIMENTAL RESULTS WITH ADFA

To verify the effectiveness of ADFA on LED, we conduct many experiments and report the results in this section. How to inject a real fault was widely studied, e.g., in [1], [34] and is out of the scope of this paper. We simulate the fault injection using computer software and calculate the solving time with the setup in Section III.F. One ADFA on LED is considered as an *instance*.

A. ADFA on LED under nibble-based fault model

For each instance, we first generate a random plaintext P , a secret key K and get the correct ciphertext C . Then, we inject one random nibble fault at Y^{30} when encrypting P

with the same K and get the faulty ciphertext C^* . Finally, we use P , C and C^* to deduce K .

To calculate the distributions of the number of candidates for f_0, f_1, f_2, f_3 in ADFA on LED, we generate random keys, plaintexts and faults 10,000 times and use the proposed DFA technique to deduce f_0, f_1, f_2, f_3 . The distributions of the number of candidates for f_0, f_1, f_2, f_3 are shown in Fig. 3. We can see that multiple candidates (1-4) of f_0, f_1, f_2, f_3 can be deduced in practice, and the numbers of the candidates for f_0, f_1, f_2, f_3 are slightly different (due to the MC function).

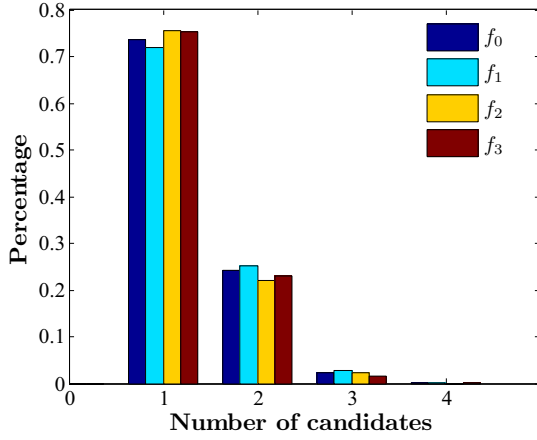


Figure 3. Distributions of the number of candidates for f_0, f_1, f_2, f_3

In ADFA on LED, we first build the algebraic equations of the full LED for the correct encryption (both P and C are fed into the equations) and equations of the last 3 rounds for the faulty encryption (only C^* is fed into the equations). Next, we deduce the fault differences ΔX^{32} with DFA and represent ΔX^{32} , ΔY^{30} with algebraic equations. At the beginning, we adopt the first strategy to directly build the algebraic equations for every forward operation in LED. The full equation set takes 20,100 variables and 32,450 ANF equations on average. The size of the generated scripts is about 725K per instance. We run 100 random instances and the secret key can be recovered in 15 hours on average, which is also consistent with the results in [16]. Then, we adopt the second strategy to build the algebraic equations for the reverse operations in LED. The full equation set takes 21,131 variables and 35,389 ANF equations on average. The size of the generated scripts is about 775K per instance. We run 1000 random instances and the distribution of the time complexity of the attack is shown in Fig. 4. We can see that

- 1) the solving time seems to follow an exponential distribution (as noted in [28], [37]).
- 2) the correct key can be solved within one minute at a success rate of 63.4%, within three minutes at a success rate of 89% and within ten minutes at a success

rate of 97.2%. In fact, if we set the threshold as one hour, the success rate is 99.48% in our experiments.

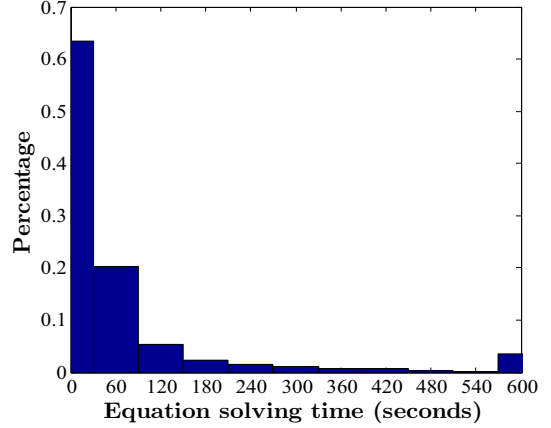


Figure 4. The distribution of time on solving equations in ADFA on LED

The comparisons of ADFA on LED with previous work are shown in Table I. As [13] provided no results and details on the experiments of DFA on LED, we only compare our result with [15], [16], [22]. Compared with [22], our attack requires less fault injections. Compared with [15], the CryptoMiniSat solver in ADFA can automatically solve for key and output the full key at a time. So no extra search on the master key is required, as in [16]. The setup of our ADFA is less costly than the 2.1 GHz Opteron workstation having 48 GB RAM [15] and the workstation having eight 3.5 GHz Xeon processors and 50 GB of RAM [16]. Meanwhile, the time of the full attack is also comparable to 45 seconds for the preprocessing phase of the attack, without considering the key enumerating phase in [15]. The time needed for our attack is less than the 14.67 hours in [15]. So AFA on LED can be more efficient than DFA. The experimental results also indicate that leveraging the algebraic equations for reverse operations in LED can significantly improve the efficiency of ADFA on LED.

B. Applications with other fault models

With the additional advantages of algebraic attacks, ADFA is much more generic than the traditional DFA. We are also interested in extending ADFA on LED to other fault models where traditional DFAs are difficult to work. More specifically, two fault models are considered.

1) Byte-based fault model in the 30th round

Under this model, the fault propagation pattern is much more complicated than that under the nibble-based fault model. According to the design of LED, the propagated fault is overlapped in the last round and it is difficult to derive ΔX^{32} from ΔY^{32} with the DFA in [13] and [15]. In our ADFA attack, we just need to build the algebraic equations

of the full LED for the correct encryption (both P and C are fed into the equations), equations of the last 3 rounds for the faulty encryption (only C^* is fed into the equations), and equations of ΔY^{30} . We run 100 random instances. On average, the solver can output the 64-bit master key within one hour.

2) Diagonal-based fault model in the 30th round

Diagonal-based fault model is a classic fault model used in DFA on AES, where the fault is induced into the diagonals (one to four matrix elements) of the state matrix [29]. Let $S = s_0 || s_1 || \dots || s_{15}$ denote the state matrix in LED. A diagonal is a set of four elements of S , where the i th diagonal D_i is defined as: $D_0 = s_0, s_5, s_{10}, s_{15}$, $D_1 = s_1, s_6, s_{11}, s_{12}$, $D_2 = s_2, s_7, s_8, s_{13}$, $D_4 = s_3, s_4, s_9, s_{14}$. Under the assumptions of injecting one diagonal fault (one to four nibbles become faulty), we run the ADFA instances for about 100 times. On average, one hour is enough to break LED.

C. Applications with LED-128

LED-128 uses a 128-bit key which is split into two 64-bit keys, K_1 and K_2 . The two halves are used alternately as round keys. Since K_1 and K_2 are independent from each other, a straightforward application of DFA in [13] and [15] cannot work on LED. How to conduct efficient fault attacks on LED-128 is an open problem.

We find that if the adversary has the capability of injecting two nibble based faults on LED-128, it is possible to break LED-128 with ADFA. The first fault is injected on one nibble in the SC output in the 46-th round, and the second fault is injected on one nibble in the SC output in the 44-th round. We build algebraic equations with ADFA in Section III and solve it with CryptoMiniSat. If 12 key bits are known, the solver can recover the full 128-bit key within about 1.5 hours.

V. IMPROVED EVALUATIONS OF DFA ON LED

Evaluating the reduced key search space in DFA for a given fault model is very important for testing the resistances of ciphers against fault attacks. Different results on evaluating the key search space of DFA on LED are provided in [13], [15]. How to find out the exact results is the main motivation of this section. Two different approaches are considered in this paper. The first is the DFA approach with an improved evaluation algorithm and the second is a modified ADFA approach with an automatic SAT solver.

A. An improved evaluation algorithm of DFA on LED

Let $\phi_{32}(K)$ and $\phi_{31}(K)$ denote the reduced key search space after analyzing the last round and the last two rounds, respectively. We write an algorithm to calculate $\phi_{32}(K)$ and $\phi_{31}(K)$. According to the known ciphertext and the design of LED, the entropy of the master key is equivalent

Algorithm 1 Compute $\phi_{32}(K)$ in DFA on LED

```

vector < int > vk32[4][4], vector < int > vk32*[4][4],
int q = {0, 3, 2, 1}
 $\Delta Y^{32} \leftarrow SR(MC^{-1}(C + C^*))$ 
for  $i = 0$  to 3 do
     $S(f_i) \leftarrow$  candidate of  $f_i$ 
end for
for  $i = 0$  to 3 do
    for  $X_i^{32} = 0$  to 15 do
        for  $X_{4+i}^{32} = 0$  to 15 do
            for  $X_{8+i}^{32} = 0$  to 15 do
                for  $X_{12+i}^{32} = 0$  to 15 do
                    for  $f_i$  in  $S(f_i)$  do
                        if  $(\Delta Y_i^{32} == (S[X_i^{32}] + S[X_i^{32} + M[i] \cdot f_{q[i \% 4]}]))$ 
                        and  $(\Delta Y_{4+i}^{32} == (S[X_{4+i}^{32}] + S[X_{4+i}^{32} + M[4+i] \cdot$ 
                         $(f_{q[i \% 4]})) \Delta Y_{8+i}^{32} == (S[X_{8+i}^{32}] + S[X_{8+i}^{32} +$ 
                         $M[8+i] \cdot f_{q[i \% 4]}))$  and  $(\Delta Y_{12+i}^{32} ==$ 
                         $(S[X_{12+i}^{32}] + S[X_{12+i}^{32} + M[12+i] \cdot f_{q[i \% 4]})) \{$ 
                            Push  $X_i^{32}, X_{4+i}^{32}, X_{8+i}^{32}, X_{12+i}^{32}$  to  $vk32[i]$ 
                            Push  $X_i^{32} + M[i] \cdot f_{q[i \% 4]}, X_{4+i}^{32} + M[4+i] \cdot$ 
                             $f_{q[i \% 4]}, X_{8+i}^{32} + M[8+i] \cdot f_{q[i \% 4]}, X_{12+i}^{32} +$ 
                             $M[12+i] \cdot f_{q[i \% 4]}$  to  $vk32*[i]\}$ 
                        end for
                    end for
                end for
            end for
        end for
    end for
     $\phi_{32}(K) = \text{sizeof}(vk32[0][0]) \times \text{sizeof}(vk32[1][0]) \times$ 
     $\text{sizeof}(vk32[2][0]) \times \text{sizeof}(vk32[3][0])$ 

```

to calculating the search space of X^{32} . The pseudocode of calculating $\phi_{32}(K)$ is shown in Algorithm 1.

The main idea of Algorithm 1 is to utilize the incomplete avalanche effect in the last round and to calculate four groups of X_i^{32} one by one, each group containing four nibbles of X_i^{32} . We generate random keys, plaintexts and faults 10,000 times and use Algorithm 1 to calculate the value of $\phi_{32}(K)$. The distributions of $\phi_{32}(K)$ are shown in Fig. 5. We can see that the value of $\phi_{32}(K)$ seems to follow a *normal distribution*. After analyzing the fault propagation of the last round, $\phi_{32}(K)$ can be reduced to $2^{17} \sim 2^{43}$ ($2^{26.6}$ on average). This is larger than the theoretical value of 2^{16} in [13], which is based on the main idea of DFA on AES. We believe that it is because the remaining key probability of analyzing the input and output fault difference for one S-Box of LED is much more than 2^{-3} in [13], which is different from the approximate value of 2^{-6} for 8-bit S-Box of AES [34]. Thus, one should be careful when applying the technique of DFA on AES to LED.

We also extend the fault analysis to the 30th and 31st round, and use the fact that only one nibble in Y^{30} becomes

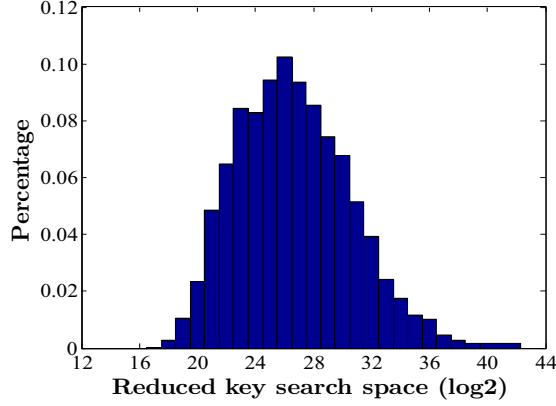


Figure 5. Reduced key search space of DFA on LED by analyzing the last round using Algorithm 1

faulty as the key filter. The key search space for the master key, $\phi_{31}(K)$, can be further reduced, as shown in Algorithm 2. We run the 100 instances, $\phi_{31}(K)$ can be reduced to an average of $2^{12.9}$. Using Algorithm 2, if the value of $\phi_{32}(K)$ is less than 2^{32} (80% probability), $\phi_{31}(K)$ can be calculated within 10 minutes. In worst cases, e.g., $\phi_{32}(K) = 2^{40}$, the time required is about 24 hours.

Algorithm 2 Compute $\phi_{31}(K)$ in DFA on LED

```

vector < int > vk31[4][4]
for  $X_0^{32}, X_4^{32}, X_8^{32}, X_{12}^{32}$  in vk32[0] do
  for  $X_1^{32}, X_5^{32}, X_9^{32}, X_{13}^{32}$  in vk32[1] do
    for  $X_2^{32}, X_6^{32}, X_{10}^{32}, X_{14}^{32}$  in vk32[2] do
      for  $X_3^{32}, X_7^{32}, X_{11}^{32}, X_{15}^{32}$  in vk32[3] do {
        Compute  $X_i^{32*}$  from vk32*[4][4]
         $Y^{30} = SR^{-1}(MC^{-1}(AC^{31-1}(SC^{-1}(SR^{-1}(MC^{-1}(AC^{32-1}(X^{32})))))))$ 
         $Y^{30*} = SR^{-1}(MC^{-1}(AC^{31-1}(SC^{-1}(SR^{-1}(MC^{-1}(AC^{32-1}(X^{32*})))))))$ 
         $\Delta Y^{30} = Y^{30} + Y^{30*}$ 
        int nzero=number of zeros in  $\Delta Y^{30}$ 
        if(nzero == 15)
          Push  $X^{32}$  to vk31 }
      end for
    end for
  end for
end for
 $\phi_{31}(K) = \text{sizeof}(vk31[0][0])$ 

```

B. Evaluating DFA on LED with ADFA

Besides the traditional DFA approach, we are also interested in finding an automatic way to evaluating DFA on LED, which was not studied in previous fault analysis work. We provide a modified ADFA approach to compute

the solutions for the secret key, which can be conducted with an automatic solver (e.g., the SAT-based solver). Compared with the manual approach in DFA, the solver approach in ADFA is much more reliable and robust.

In the attack, we only build the algebraic equations for the last 3 rounds of LED (both the correct and the faulty encryption) and the fault difference ($\Delta Y^{32}, \Delta X^{32}, \Delta Y^{30}$). The values of C and C^* are also fed into the solver. Compared with the ADFA approach in Section III and Section IV (including the equation set for a full correct LED encryption), the number of the equations in this problem is much smaller and the value of the plaintext P is not required.

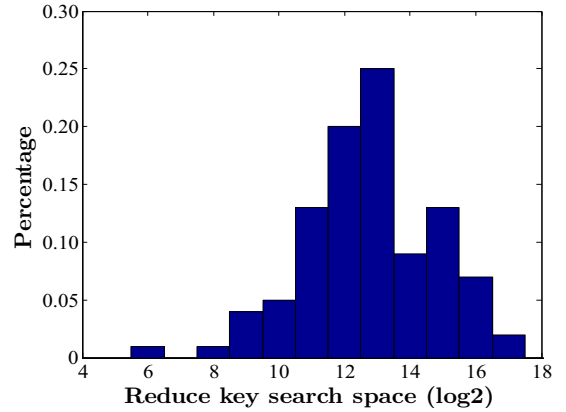


Figure 6. The reduced key search space of ADFA on LED under nibble-based fault model

In this scenario, there are multiple solutions for these equations. The original CryptoMiniSat solver always outputs a satisfiable but wrong key and stops, which makes the attack fail. Fortunately, since CryptoMiniSat 2.9.4, it can output multiple solutions for the input variables. For each solution, the original solver always outputs the values of all the input variables and writes them to the result file. As the number of all the boolean variables (about 3011) is larger than the number of the key variables (80), the original solver cannot finish in a reasonable amount of time. Meanwhile, it cannot calculate the number of all the possible solutions.

With the help of M. Soos, an author of CryptoMiniSat, we manage to modify the source code of the solver. The modified solver can automatically count and output all the possible key solutions for ADFA on LED. We run 100 instances and calculate the distribution of the numbers of the possible solutions for the master key, as shown in Fig. 6. The key search space of LED is reduced to $2^6 \sim 2^{17}$ ($2^{12.9}$ on average), which is highly consistent with the results in Section V.A. Since less variables and equations are fed to the solver, the time complexity required in this section is even less than that in Section IV.A.

C. Application with byte-based fault model

The traditional approaches of evaluating DFA on LED requires manual analysis of the fault propagation path, which is difficult under complicated fault models. We take evaluating the key search space of DFA on LED under the byte-based fault model in the 30th round as an example. Since the propagated fault is overlapped in the last round, it is difficult to compute the key search space with DFA. Algorithm 1 does not work in this case. As to ADFA approach, all we needed is to change the algebraic equations of representing ΔY^{30} and delete the equations of representing ΔX^{32} .

Firstly, assuming the first byte of Y^{30} is faulty, we run the instances of ADFA on LED for about 100 times. The distribution of the numbers of the possible solutions for DFA on LED is shown in Fig. 7. It is interesting to notice that, for this fault location, the key search space of LED can be reduced to $2^6 \sim 2^{14}$ ($2^{9.9}$ on average), which is even smaller than that in nibble-based fault model.

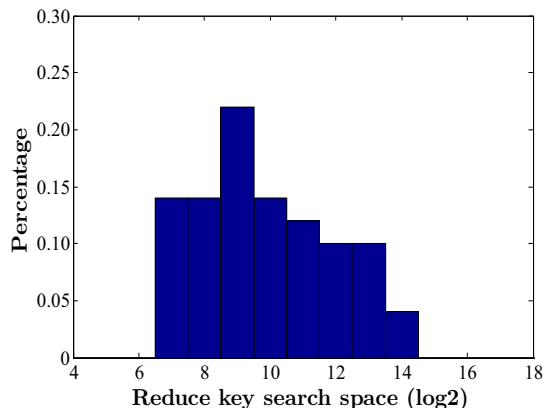


Figure 7. The reduced key search space of ADFA on LED under byte-based fault model

Then, we launch ADFA on LED to all the possible fault locations under both nibble-based and byte-based fault model. For each location, we run the instances for about 100 times and calculate the average reduced key search space, as shown in Fig. 8. The result is consistent with that in Fig. 7, which shows that the reduced key search space of DFA on LED ($2^{10.1}$ on average for eight fault locations) under byte-based fault model is smaller than that under nibble-based fault model ($2^{12.9}$ on average for sixteen fault locations).

VI. CONCLUSION

This paper proposes a fault attack technique on LED by combining algebraic cryptanalysis and differential fault analysis (DFA), named algebraic DFA (ADFA). We show that the secret key of LED can be recovered by only one fault injection with lower cost and time complexity than previous work [15], [16]. In addition, we provide two different approaches to evaluate the reduced key search space of DFA on LED. An improved evaluation algorithm of DFA on

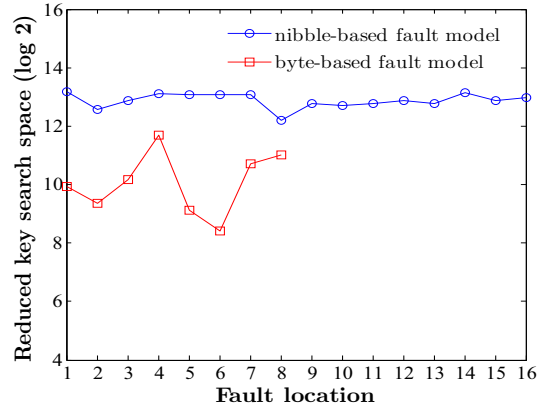


Figure 8. The reduced key search space of ADFA on LED for different fault locations

LED is proposed. Meanwhile, a modified ADFA approach is also provided to calculate the solutions for the secret key. The results of the two approaches are consistent and more accurate than previous work [13], [15]. We also successfully extend ADFA on LED to other complicated fault models using a single fault injection, where traditional DFAs are difficult to launch. The results of this paper show that ADFA is efficient and generic. We hope that our work enriches the understanding of algebraic fault analysis of block ciphers and helps to evaluate the security of block ciphers against fault attacks.

ACKNOWLEDGMENT

The authors would like to thank Ruilin Li, Zheng Gong, Siwei Sun and the anonymous referees for helpful discussions and comments. This work was supported in part by the National Natural Science Foundation of China under the grants 60772082, 61173191, 61272491, and the US National Science Foundation under the grant CNS-0644188.

REFERENCES

- [1] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, C. Whelan. The Sorcerers Apprentice Guide to Fault Attacks. In IEEE 94, pp. 370-382, 2006.
- [2] I. Biehl, B. Meyer, V. Muller. Differential fault analysis on elliptic curve cryptosystems. In Proceedings of CRYPTO 2000. LNCS, vol. 1880, pp. 131-146, 2000.
- [3] E. Biham, A. Shamir. Differential Fault Analysis of Secret Key Cryptosystem. In Proceedings of CRYPTO 1997, LNCS, vol. 1294, pp. 513-525, 1997.
- [4] A. Bogdanov, L.R. Knudsen, G. Leander, et al. PRESENT: An Ultra-Lightweight Block Cipher. In Proceedings of CHES 2007. LNCS, vol. 4727, pp. 450-466, 2007.
- [5] D. Boneh, R.A. DeMillo, R.J. Lipton. On the Importance of Checking Cryptographic Protocols for Faults. In Proceedings of EUROCRYPT 1997, LNCS, vol. 1233, pp. 37-51, 1997.

- [6] N. Courtois, J. Pieprzyk. Cryptanalysis of Block Ciphers with Overdefined Systems of Equations. In Proceedings of ASIACRYPT 2002, LNCS, vol. 2501, pp. 267-287, 2002.
- [7] N. Courtois, D. Ware, K. Jackson. Fault-Algebraic Attacks on Inner Rounds of DES. In Proceedings of eSmart 2010, pp. 22-24, 2010.
- [8] J. Ding, J. Buchmann, M.S.E. Mohamed, et al. MutantXL algorithm. In Proceedings of the 1st International Conference in Symbolic Computation and Cryptography, pp. 16-22, 2008.
- [9] J.-C. Faugère, Gröbner Bases. Applications in Cryptology. In Proceedings of FSE 2007 Invited Talk, available at: <http://fse2007.uni.lu/slides/faugere.pdf>.
- [10] J. Guo, T. Peyrin, A. Poschmann, and M. Robshaw. The LED Block Cipher. In Proceedings of CHES 2011, LNCS, vol. 6917, pp. 326-341, 2011.
- [11] J.J. Hoch, A. Shamir. Fault analysis of stream ciphers. In Proceedings of CHES 2004, LNCS, vol. 3156, pp. 240-253, 2004.
- [12] M. Hojsik and B. Rudolf. Differential fault analysis of Trivium. In Proceedings of FSE 2008, LNCS, vol. 5086, pp. 58-172, 2008.
- [13] K. Jeong and C. Lee. Differential Fault Analysis on Block Cipher LED-64. Future Information Technology, Application, and Service, LNEE, vol. 164, pp.747-755, 2012.
- [14] K. Jeong, Y. Lee, J. Sungb, S. Honga. Differential fault analysis on block cipher SEED. Mathematical and Computer Modelling vol. 55, pp. 26-34, 2012.
- [15] P. Jovanovic, M. Kreuzer, and I. Polian. A Fault Attack on the LED Block Cipher. In Proceedings of COSADE 2012, LNCS, vol. 7275, pp. 120-134, 2012.
- [16] P. Jovanovic, M. Kreuzer and I. Polian, An Algebraic Fault Attack on the LED Block Cipher. Cryptology ePrint Archive. Available: <http://eprint.iacr.org/2012/400.pdf>, 2012.
- [17] L.R. Knudsen, C.V. Miolane. Counting equations in algebraic attacks on block ciphers. International Journal of Information Security, vol. 9, No. 2, pp. 127-135, 2010.
- [18] P. Kocher. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In Proceedings of CRYPTO 1996, LNCS, vol. 1109, pp. 104-113, 1996.
- [19] P. Kocher, J. Jaffe, B. Jun. Differential power analysis. In Proceedings of CRYPTO 1999, LNCS, vol. 1666, pp. 388-397, 1999.
- [20] W. Li, D.W. Gu, J.R. Li. Differential fault analysis on the ARIA algorithm. Information Sciences. 2008: 3727-3737.
- [21] W. Li, D.W. Gu, J.R. Li, et. al. Differential fault analysis on Camellia. The Journal of Systems and Software, 2011, vol. 83(5): 844-851.
- [22] W. Li, D.W. Gu, et al. Security Analysis of the LED Lightweight Cipher in the Internet of Things. Chinese Journal of Computers, 2012, 35(3): 434-445.
- [23] M. Mohamed, W.S.A.E Mohamed, J. Ding, and J. Buchmann. MXL2: Solving Polynomial Equations over GF(2) using an Improved Mutant Strategy. In Proceedings of the Second international Workshop on Post-Quantum Cryptography, LNCS, vol. 5299, pp. 20-215, 2008.
- [24] M. Mohamed, S. Bulygin and J. Buchmann. Improved Differential Fault Analysis of Trivium. In COSADE 2011, pp. 147-158, 2011.
- [25] M. Mohamed, S. Bulygin, M. Zohner, A. Heuser, M. Walter. Improved Algebraic Side-Channel Attack on AES. Cryptology ePrint Archive. Available: <http://eprint.iacr.org/2012/084.pdf>, 2012.
- [26] G. Piret, J.J. Quisquater. A Differential Fault Attack Technique against SPN Structures, with Application to the AES and Khazad. In Proceedings of CHES 2003, LNCS, vol. 2779, pp. 77-88, 2003.
- [27] J. J. Quisquater, D. Samyde. A new tool for non-intrusive analysis of smart cards based on electro-magnetic emissions: the SEMA and DEMA methods, Eurocrypt rump session, 2000.
- [28] M. Renauld, F.-X. Standaert. Algebraic Side-Channel Attacks. In Proceedings of INSCRYPT 2009, LNCS, vol. 6151, pp. 393-410, 2009.
- [29] D. Saha, D. Mukhopadhyay, and D. RoyChowdhury, A Diagonal Fault Attack on the Advanced Encryption Standard. Cryptology ePrint Archive. Available: <http://eprint.iacr.org/2009/581.pdf>, 2009.
- [30] SAT. SAT Race Competition. <http://www.satcompetition.org/>.
- [31] K. Shibutani, T. Isobe, H. Hiwatari, et al. Piccolo: An Ultra-Lightweight Blockcipher. In Proceedings of CHES 2011, LNCS, vol. 6917, pp. 342-357, 2011.
- [32] M. Soos, K. Nohl, and C. Castelluccia. Extending SAT Solvers to Cryptographic Problems. In Proceedings of SAT 2009, LNCS, vol. 5584, pp. 244-257, 2009.
- [33] J. Takahashi and T. Fukunaga. Improved Differential Fault Analysis on CLEFIA. In Proceedings of FDTC 2008, pp. 25-34, 2008.
- [34] M. Tunstall, D. Mukhopadhyay, S. Ali. Differential fault analysis of the advanced encryption standard using a single fault. In Proceedings of WISTP 2011, LNCS, vol. 6633, pp. 224-233, 2011.
- [35] F. Zhang, X.J. Zhao, S.Z. Guo, et al. Improved Algebraic Fault Analysis: A Case Study on Piccolo and Applications to Other Lightweight Block Ciphers. In Proceedings of COSADE 2013, LNCS, vol. 7864, pp. 62-79, 2013.
- [36] X.J. Zhao and T. Wang. Further Improved Differential Fault Analysis on Camellia by Exploring Fault Width and Depth. Cryptology ePrint Archive, <http://eprint.iacr.org/2010/026.pdf>, 2010.
- [37] X.J. Zhao, S.Z. Guo, F. Zhang, et al. MDASCA: An Enhanced Algebraic Side-Channel Attack for Error Tolerance and New Leakage Model Exploitation. In Proceedings of COSADE 2012, LNCS, vol. 7275, pp. 231-248, 2012.